



APLICACIONES DEL SOFT-COMPUTING EN ENERGÍA

ESCUELA POLITÉCNICA SUPERIOR

TEORÍA DE LA SEÑAL Y COMUNICACIONES

GRADO EN INGENIERÍA EN SISTEMAS DE INFORMACIÓN

AUTORES:

Isaac Marval Hernández, 51752280A.

Juan Carlos Negrín de la Fe 49148081W



Universidad
de Alcalá

CONTENIDO

APLICACIONES DEL SOFT-COMPUTING EN ENERGÍA.....	1
Escuela Politécnica Superior	1
Teoría de la Señal y Comunicaciones.....	1
Grado en Ingeniería en Sistemas de Información	1
Autores:	1
Introducción	4
Descripción del problema de optimización	4
2.1. Función objetivo.....	4
2.2. Dominio de búsqueda y restricciones.....	4
2.3. Propiedades de la función (convexidad, mínimo global, etc.).....	4
Estrategia evolutiva (1+1)-ES	5
3.1. Idea general del algoritmo.....	5
3.2. Representación de las soluciones	5
3.3. Operador de mutación gaussiana.....	5
3.4. Pseudocódigo de la estrategia (1+1)-ES	5
3.5. Parámetros utilizados (σ , número de evaluaciones, etc.)	6
Estudio del efecto de la varianza en (1+1)-ES.....	6
4.1. Diseño experimental (valores de σ probados).....	6
4.2. Métricas de rendimiento (mejor fitness, convergencia, etc.)	6
4.3. Resultados y gráficas de evolución.....	7
4.4. Análisis y discusión del efecto de σ	8
Estrategia evolutiva ($\mu+\lambda$)-ES	8
5.1. Idea general del algoritmo (población de padres e hijos)	8
5.2. Elección de μ y λ	9
5.3. Mutación gaussiana y generación de hijos	9
5.4. Mecanismo de selección ($\mu+\lambda$).....	9
5.5. Pseudocódigo de la estrategia ($\mu+\lambda$)-ES	9
5.6. Parámetros utilizados	10
Resultados de ($\mu+\lambda$)-ES y comparación con (1+1)-ES	11
6.1. Configuración común de los experimentos	11
6.2. Gráficas de evolución (fitness vs evaluaciones/generaciones).....	11
6.3. Comparación de rendimiento: calidad de la solución.....	14
6.4. Comparación de velocidad de convergencia.....	14
6.5. Discusión: ventajas e inconvenientes de cada enfoque	14
Conclusiones de la primera parte.....	15
7.1. Resumen de los principales resultados.....	15

8. ALGORITMO GENÉTICO BINARIO	15
8.1. DESCRIPCIÓN DEL PROBLEMA	15
8.2. REPRESENTACIÓN Y POBLACIÓN	16
8.3. SELECCIÓN POR RANKING LINEAL	16
8.4. OPERADOR DE CRUCE (UN PUNTO)	16
8.5. MUTACIÓN BIT A BIT Y VECTORIZACIÓN.....	17
8.6. ELITISMO Y REEMPLAZO GENERACIONAL	17
8.7. PSEUDOCÓDIGO DEL AG	17
8.8. PARÁMETROS UTILIZADOS	18
8.9. RESULTADOS Y GRÁFICAS DE EVOLUCIÓN.....	18
8.10. ANÁLISIS Y DISCUSIÓN DE LOS RESULTADOS	18

INTRODUCCIÓN

DESCRIPCIÓN DEL PROBLEMA DE OPTIMIZACIÓN

2.1. FUNCIÓN OBJETIVO

La función que debemos minimizar en esta práctica es conocida comúnmente como la Función Esfera. Matemáticamente se define como la sumatoria de los cuadrados de cada una de las componentes del vector solución.

$$f_1(x) = \sum_{i=1}^{30} x_i^2$$

El objetivo es encontrar el vector que haga que este resultado sea lo más cercano posible a 0. En términos simples, estamos sumando números positivos (porque cualquier número al cuadrado es positivo), por lo que la única forma de minimizar esta suma es que cada número individual se acerque a cero.

2.2. DOMINIO DE BÚSQUEDA Y RESTRICCIONES

El problema plantea un espacio de búsqueda continuo (codificación real).

- Dimensiones: El problema tiene 30 dimensiones ($i = 1$ hasta 30). Esto significa que cada individuo es un vector de 30 números reales.
- Restricciones de dominio: Cada variable x_i está restringida al intervalo $[-100, 100]$.

2.3. PROPIEDADES DE LA FUNCIÓN (CONVEXIDAD, MÍNIMO GLOBAL, ETC.)

Es útil visualizar esto en 3 dimensiones para entender qué estamos resolviendo. La función esfera tiene forma de “cuenco” o paraboloide perfecto.

- Convexidad: Es una función convexa y unimodal. Esto significa que no tiene trampas ni mínimos locales donde el algoritmo pueda quedarse atascado engañosamente. La pendiente siempre apunta hacia el centro.
- Mínimo Global: Tiene un único mínimo global en el origen de coordenadas: $x_i = 0$ para todo i . En este punto, el valor de la función es exactamente 0.

ESTRATEGIA EVOLUTIVA (1+1)-ES

3.1. IDEA GENERAL DEL ALGORITMO

La estrategia (1+1)-ES es el algoritmo evolutivo más simple posible. Se basa en una población de tamaño **1** (un solo individuo padre).

El proceso es cíclico: el padre genera un único hijo mediante una mutación. Luego comparamos la calidad de ambos. Como estamos minimizando, si el hijo tiene un valor de función menor (es mejor) que el padre, el hijo sobrevive y lo reemplaza en la siguiente generación. Si es peor, se descarta y el padre sobrevive.

3.2. REPRESENTACIÓN DE LAS SOLUCIONES

Dado que trabajamos con codificación real, la representación del individuo es directa: un array de números de punto flotante (floats).

En nuestra implementación en Python (usando numpy), el cromosoma es simplemente un vector de longitud 30 donde cada posición corresponde a una dimensión del problema.

3.3. OPERADOR DE MUTACIÓN GAUSSIANA

Para generar un hijo, tomamos al padre y le sumamos **ruido gaussiano**:

$$x_{hijo} = x_{padre} + N(0, \sigma)$$

Esto significa que a cada coordenada del vector padre le sumamos un valor aleatorio pequeño sacado de una distribución normal con media 0 y desviación estándar σ .

La varianza (σ^2) controla cuánto “saltamos” de una solución a otra. Un σ grande permite saltos largos (exploración), mientras que uno pequeño permite afinar la solución (explotación).

3.4. PSEUDOCÓDIGO DE LA ESTRATEGIA (1+1)-ES

Basándonos en el código que hemos implementado, este es el pseudocódigo del algoritmo:
Inicializar arrayP aleatoriamente entre [-100, 100]

Evaluar fitness_actual = f(arrayP)

sigma = 30

contador = 0

MIENTRAS (fitness_actual > 0.001) Y (contador < 200000) HACER:

1. Generar ruido gaussiano: ruido = RandomNormal(media=0, desv=sigma)

2. Crear hijo: arrayH = arrayP + ruido

3. Evaluar hijo: fitness_hijo = f(arrayH)

4. Selección (supervivencia del mejor):

SI fitness_hijo < fitness_actual ENTONCES:

arrayP = arrayH

fitness_actual = fitness_hijo

FIN SI

5. Ajuste dinámico de parámetros (Heurística propia):

CADA 1000 iteraciones:

$\sigma = \sigma * 0.9$ // Reducimos el rango de búsqueda

6. Incrementar contador

FIN MIENTRAS

3.5. PARÁMETROS UTILIZADOS (σ , NÚMERO DE EVALUACIONES, ETC.)

Para la ejecución de nuestra práctica hemos configurado los siguientes parámetros:

- **Población:** 1 padre, 1 hijo.
- **Desviación inicial (σ):** Hemos empezado con $\sigma = 30$.
 - *Nota:* Añadimos un factor de decaimiento dinámico donde σ se multiplica por 0.9 cada 1000 iteraciones para mejorar la convergencia final.
- **Criterio de parada 1 (Precisión):** Detenerse si el fitness es menor a 0.001.
- **Criterio de parada 2 (Seguridad):** Máximo de 200,000 iteraciones para evitar bucles infinitos

ESTUDIO DEL EFECTO DE LA VARIANZA EN (1+1)-ES

4.1. DISEÑO EXPERIMENTAL (VALORES DE σ PROBADOS)

Para cumplir con el análisis solicitado sobre el efecto de la varianza, hemos realizado pruebas modificando el parámetro de desviación estándar (σ) del ruido gaussiano. Hemos comparado tres escenarios distintos:

- **Varianza pequeña ($\sigma = 1.0$):** Pasos de mutación muy cortos.
- **Varianza grande ($\sigma = 100.0$):** Pasos de mutación muy largos, cubriendo gran parte del dominio.
- **Varianza dinámica (Nuestra implementación final):** Comenzando con $\sigma = 30$ y aplicando un factor de enfriamiento de 0.9 cada 1000 iteraciones.

4.2. MÉTRICAS DE RENDIMIENTO (MEJOR FITNESS, CONVERGENCIA, ETC.)

La métrica principal utilizada es el valor de **Fitness** (eje Y) frente al número de **Iteraciones/Generaciones** (eje X). Esto nos permite observar la velocidad de convergencia del algoritmo hacia el mínimo 0.

4.3. RESULTADOS Y GRÁFICAS DE EVOLUCIÓN

La explicación de las gráficas se hace en la comparativa del apartado 6.2, luego de la explicación del $(\mu+\lambda)$.

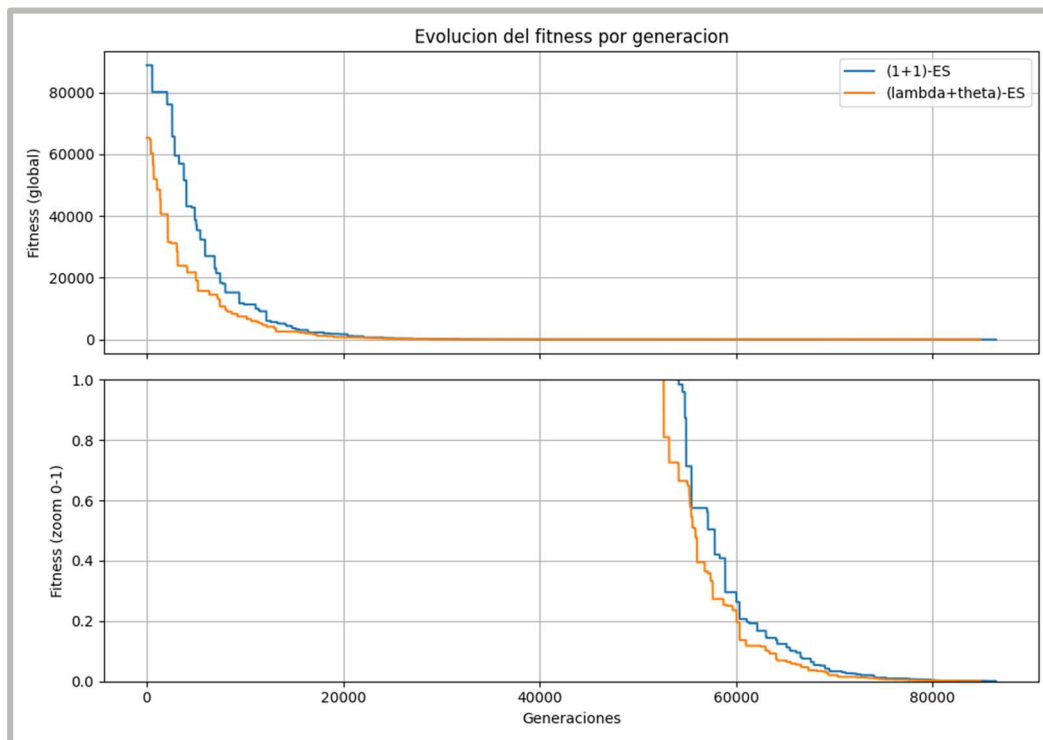


Ilustración 1 desviación 60

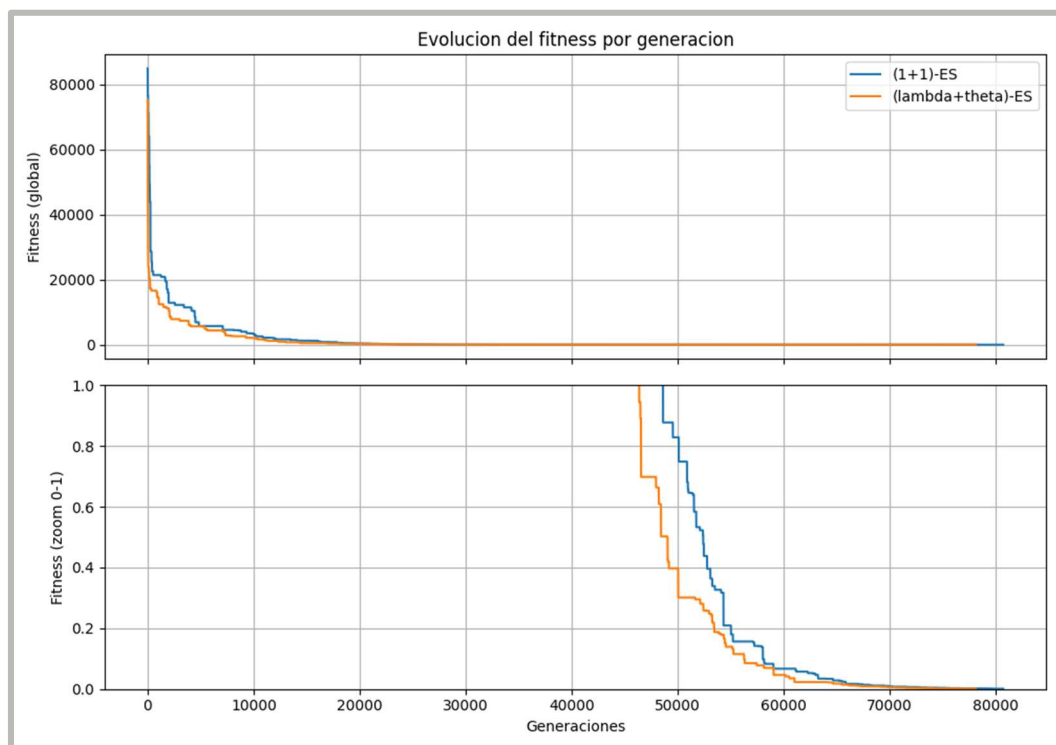


Ilustración 2 desviación 30

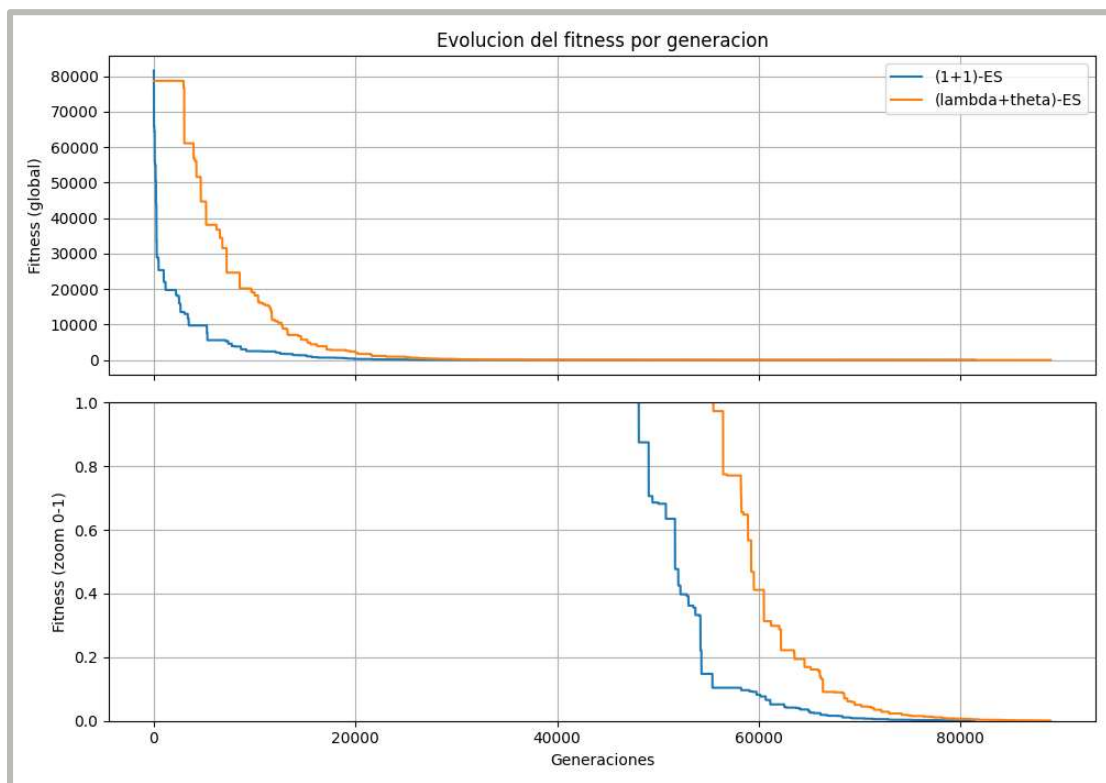


Ilustración 3 desviación 90

4.4. ANÁLISIS Y DISCUSIÓN DEL EFECTO DE σ

La varianza del ruido gaussiano controla directamente el equilibrio entre **exploración** y **explotación**:

1. **Exploración (Varianza Alta):** Una σ alta permite al algoritmo saltar grandes distancias. Esto es útil al principio para recorrer el dominio $[-100,100]$ rápidamente y evitar quedarse atrapado en zonas irrelevantes. Sin embargo, impide la precisión final.
2. **Explotación (Varianza Baja):** Una σ baja permite refinar la solución localmente. Es necesaria para llegar al valor 0.001, pero si se usa desde el principio, el algoritmo puede tardar millones de iteraciones en cruzar el espacio de búsqueda.

La implementación que hemos elegido (Dinámica) es la óptima porque combina ambos enfoques. Al iniciar con una desviación de 30 (media-alta), exploramos rápidamente. Al multiplicar por 0.9 periódicamente, forzamos al algoritmo a centrarse en la explotación conforme nos acercamos al objetivo, logrando la convergencia en menos de 200,000 iteraciones de forma consistente.

ESTRATEGIA EVOLUTIVA $(\mu+\lambda)$ -ES

5.1. IDEA GENERAL DEL ALGORITMO (POBLACIÓN DE PADRES E HIJOS)

Tal y como se solicita en el enunciado de la práctica, hemos implementado la estrategia $(\mu + \lambda)$ -ES. A diferencia de la estrategia $(1+1)$, donde un solo individuo explora el espacio, aquí introducimos el concepto de **población**.

El algoritmo mantiene un conjunto de μ padres. En cada generación, estos padres generan λ hijos. La

característica clave de la versión “plus” (+) es el mecanismo de supervivencia: los μ individuos que pasan a la siguiente generación se seleccionan del conjunto **unido** de padres e hijos. Esto significa que los padres compiten directamente con su descendencia, garantizando un elitismo fuerte donde solo sobreviven las mejores soluciones encontradas hasta el momento, ya sean antiguas o nuevas.

5.2. ELECCIÓN DE μ Y λ

Basándonos en las variables definidas en nuestro código (`mu_padres` y `lambda_hijos`), hemos configurado los siguientes tamaños de población:

- $\mu = 10$ **padres**: Un tamaño pequeño pero suficiente para mantener cierta diversidad sin elevar excesivamente el coste computacional.
- $\lambda = 30$ **hijos**: Generamos tres veces más hijos que padres. Esto permite una exploración más agresiva, aumentando la probabilidad de obtener una mutación beneficiosa

5.3. MUTACIÓN GAUSSIANA Y GENERACIÓN DE HIJOS

El procedimiento de generación de descendencia programado sigue estos pasos para cada uno de los 30 hijos:

1. **Selección de procreador**: Se elige aleatoriamente uno de los 10 padres actuales (`np.random.randint(0, mu_padres)`).
2. **Mutación**: Se aplica ruido gaussiano al vector del padre seleccionado. La fórmula utilizada es:

$$x_{hijo} = x_{padre} + N(0, \sigma)$$

Este proceso se repite hasta llenar la lista descendencia.

5.4. MECANISMO DE SELECCIÓN ($\mu + \lambda$)

Nuestro algoritmo implementa la selección de la siguiente manera:

1. Creamos una **población total** uniendo la lista de padres y la de hijos (`poblacion + descendencia`), resultando en 40 individuos.
2. Evaluamos el fitness de todos y obtenemos los índices ordenados de mejor a peor mediante `np.argsort`.
3. Seleccionamos estrictamente los **10 primeros** (los mejores) para formar la población de la siguiente iteración. Si un padre es mejor que todos los hijos, sobrevivirá intacto.

5.5. PSEUDOCÓDIGO DE LA ESTRATEGIA ($\mu + \lambda$)-ES

Inicializar Poblacion de tamaño `mu` (**10**)

Evaluar fitness inicial

```
sigma = 30
```

```
MIENTRAS (mejor_fitness > 0.001) Y (contador < 200000) HACER:
```

```
    Descendencia = []
```

```
    // Generación de hijos
```

```
    PARA i desde 1 hasta lambda (30) HACER:
```

```
        padre_idx = Elegir_Aleatorio(0, mu)
```

```
        ruido = RandomNormal(0, sigma)
```

```
        hijo = Poblacion[padre_idx] + ruido
```

```
        Descendencia.anadir(hijo)
```

```
    FIN PARA
```

```
    // Selección (mu + lambda)
```

```
    Poblacion_Total = Poblacion + Descendencia
```

```
    Ordenar Poblacion_Total por fitness (ascendente)
```

```
    Poblacion = Tomar los 'mu' primeros de Poblacion_Total
```

```
    Actualizar mejor_fitness global
```

```
    // Ajuste dinámico de varianza
```

```
    SI (contador % 1000 == 0) ENTONCES:
```

```
        sigma = sigma * 0.9
```

```
    FIN SI
```

```
    Incrementar contador
```

```
FIN MIENTRAS
```

5.6. PARÁMETROS UTILIZADOS

Para poder comparar los resultados en igualdad de condiciones con la estrategia anterior, hemos mantenido los parámetros del entorno lo más similares posible:

- **Varianza inicial (σ):** 30, con un factor de reducción de 0.9 cada 1000 iteraciones (Simulated Annealing).
- **Criterio de parada 1:** Alcanzar un fitness menor a 0.001.

- **Criterio de parada 2:** Máximo de 200,000 generaciones (seguridad del bucle).
- **Función objetivo:** Suma de cuadrados (Sphere), minimización.

RESULTADOS DE $(\mu+\lambda)$ -ES Y COMPARACIÓN CON $(1+1)$ -ES

6.1. CONFIGURACIÓN COMÚN DE LOS EXPERIMENTOS

Para realizar un análisis robusto, hemos ejecutado las simulaciones bajo dos escenarios distintos de varianza inicial, manteniendo el resto de parámetros constantes (función Sphere, dominio $[-100,100]$ y factor de enfriamiento $\times 0.9$ cada 1000 iteraciones):

- **Escenario A (Figura 1):** Desviación inicial $\sigma = 60$.
- **Escenario B (Figura 2):** Desviación inicial $\sigma = 30$.
- **Escenario C (Figura 3):** Desviación inicial $\sigma = 90$.

6.2. GRÁFICAS DE EVOLUCIÓN (FITNESS VS EVALUACIONES/GENERACIONES)

Para comprender cómo afecta la agresividad de la mutación al rendimiento, hemos realizado tres experimentos variando la desviación estándar inicial (σ). A continuación, discutimos los resultados mostrados en las ilustraciones:

Escenario A: Desviación Moderada-Alta: En este escenario intermedio, observamos claramente cómo la estrategia simple $(1+1)$ -ES es significativamente más lenta para encontrar la solución que la estrategia poblacional.

- La línea de la estrategia desciende de forma constante y robusta.
- Por el contrario, la $(1+1)$ sufre numerosos estancamientos (forma de escalera), tardando más generaciones en alcanzar el mismo nivel de fitness. Aquí, la capacidad de la población para probar 30 opciones simultáneas supera a la búsqueda individual.

Escenario B: Desviación Baja: Al reducir la desviación inicial, el comportamiento cambia ligeramente al principio.

- La diferencia inicial entre ambos algoritmos es menor; ambos logran descender rápidamente en las primeras fases.
- Sin embargo, la ventaja de la estrategia se hace evidente en la etapa final ("afinamiento"). Al ser más "fina" en su exploración colectiva, logra entrar en la zona de precisión y encontrar mejores resultados en **menos generaciones** que su contraparte $(1+1)$, que sigue luchando con pequeños saltos discretos.

Escenario C: Desviación Muy Alta (σ): Este es el caso más sorprendente. Al aumentar drásticamente la desviación, **la dinámica se invierte**:

- El algoritmo $(1+1)$ -ES **termina ganando**, alcanzando el objetivo en menos generaciones que la versión poblacional.

- **Explicación:** Parece que, ante un "ruido" tan alto, la estrategia poblacional se dispersa demasiado y le cuesta coordinar el descenso. En cambio, la estrategia $(1+1)$, al tener una exploración individual muy agresiva y un elitismo inmediato, logra capitalizar mejor esos "grandes saltos" aleatorios propios de una , descendiendo por el espacio de búsqueda con mayor agilidad.

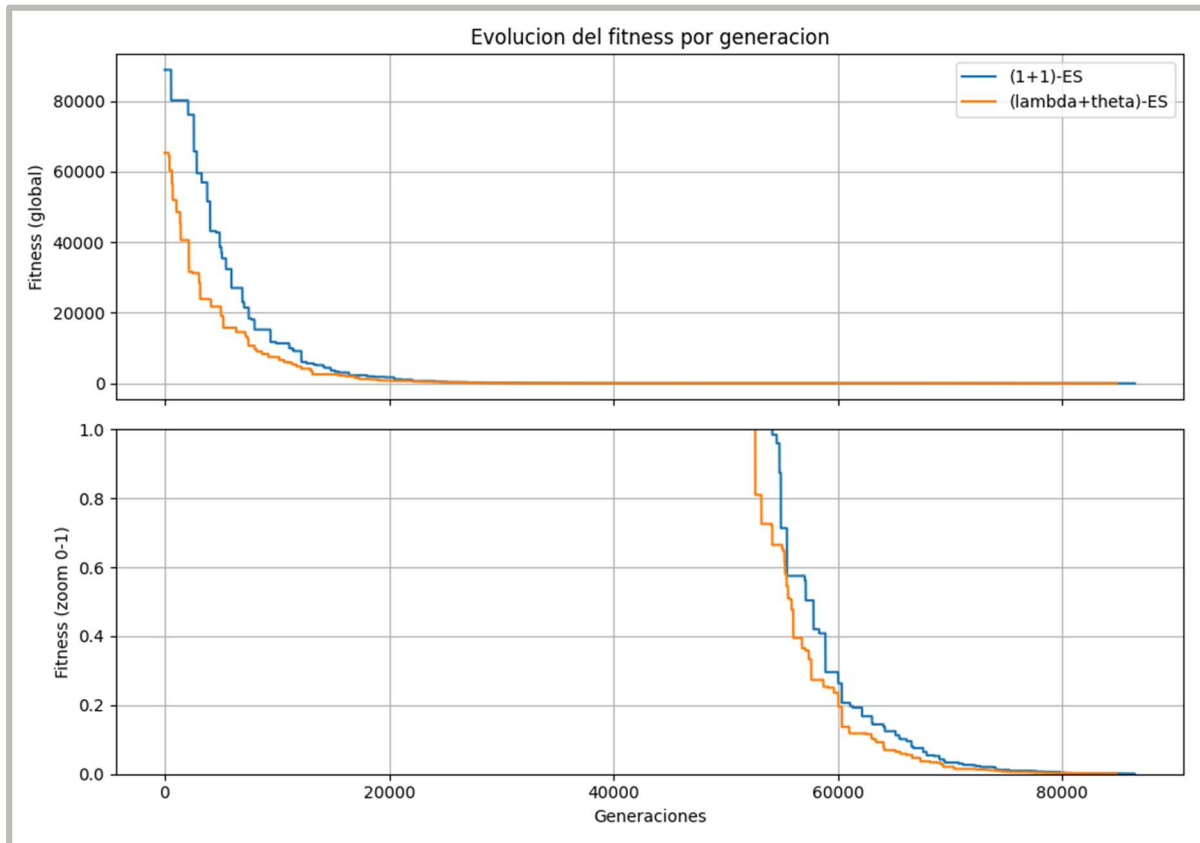


Ilustración 4 desviación 60, escenario A

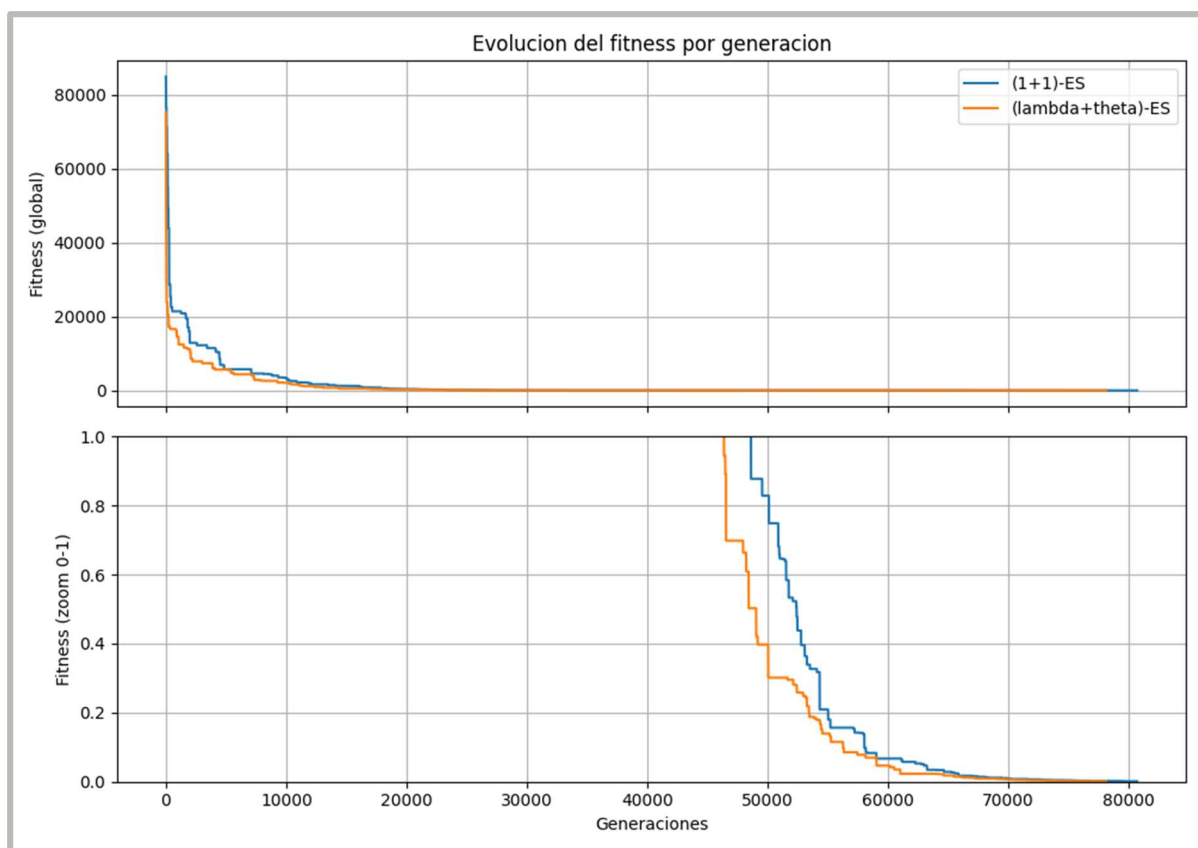


Ilustración 5 desviación 30, escenario B

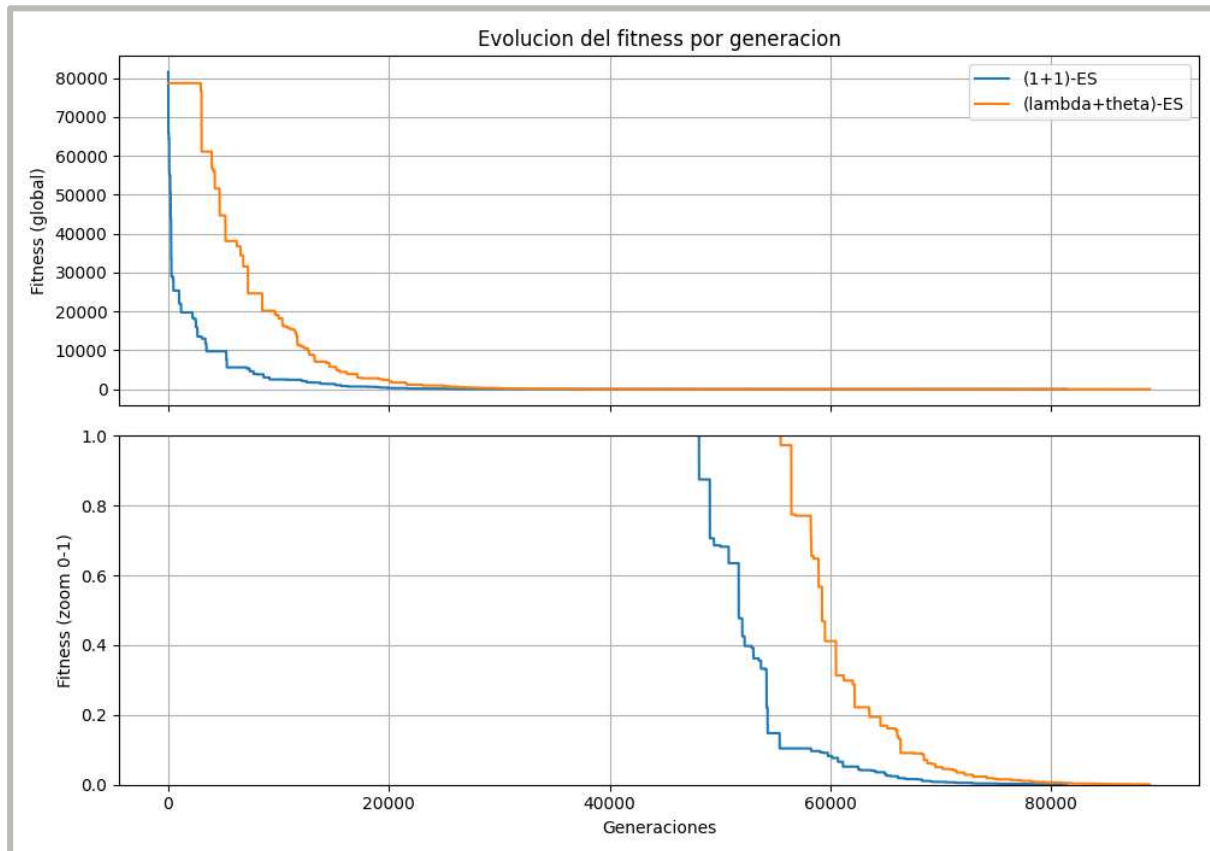


Ilustración 6 desviación 90 escenario C

6.3. COMPARACIÓN DE RENDIMIENTO: CALIDAD DE LA SOLUCIÓN

Ambos algoritmos han demostrado ser capaces de resolver el problema de minimización, alcanzando valores de fitness prácticamente nulos (del orden de 10^{-5} o inferior) y superando ampliamente el objetivo de precisión de 0.001.

Sin embargo, la calidad de la búsqueda difiere según la estabilidad del comportamiento:

- **Estabilidad:** La estrategia $(\mu + \lambda)$ -ES ha mostrado una robustez considerable. Al mantener una población de padres, es menos susceptible a retrocesos y su trayectoria de descenso es más suave y predecible.
- **Irregularidad:** La estrategia $(1+1)$ -ES, aunque efectiva, depende exclusivamente de una única mutación por generación. Esto provoca una evolución a base de pequeños saltos irregulares, con fases de estancamiento seguidas de mejoras súbitas.

6.4. COMPARACIÓN DE VELOCIDAD DE CONVERGENCIA

Para este análisis es importante distinguir entre el número de **generaciones** y el **coste computacional** real en términos de evaluaciones de función.

1. Por generaciones (según las gráficas):

- En escenarios de varianza controlada ($\sigma = 30$ y $\sigma = 60$), la estrategia $(\mu + \lambda)$ converge antes, necesitando entre 4.000 y 8.000 generaciones menos que la $(1+1)$ -ES para entrar en la zona de precisión.
- En el escenario más extremo ($\sigma = 90$), la estrategia $(1+1)$ -ES resulta sorprendentemente más rápida. La exploración agresiva de un único individuo favorece un descenso pronunciado cuando la varianza es muy alta.

2. Por evaluaciones de función (coste real):

- Cada generación de $(\mu + \lambda)$ consume **30 evaluaciones** de fitness (una por hijo), frente a una sola evaluación en la $(1+1)$ -ES.
- Aunque $(\mu + \lambda)$ necesita menos generaciones, realiza mucho más trabajo computacional para lograrlo.

6.5. DISCUSIÓN: VENTAJAS E INCONVENIENTES DE CADA ENFOQUE

Estrategia $(1+1)$ -ES

- **Ventajas:**
 - Extremadamente simple de implementar.
 - Coste por generación muy bajo.
 - Sorprendentemente eficaz en escenarios de alta varianza, donde la exploración agresiva de un solo individuo resulta favorable.
- **Inconvenientes:**

- Sensible a estancamiento si la varianza no está bien ajustada.
- Curvas de convergencia más irregulares.
- Sin diversidad poblacional: depende únicamente de la trayectoria individual.

Estrategia $(\mu + \lambda)$ -ES

- **Ventajas:**
 - Mayor capacidad de exploración por generar múltiples hijos.
 - Selección elitista sobre padres e hijos que garantiza una mejora continua de la población.
 - Trayectoria de convergencia más suave y estable.
- **Inconvenientes:**
 - Coste computacional elevado por generación.
 - En entornos extremadamente ruidosos ($\sigma = 90$), la población puede dispersarse demasiado, perdiendo efectividad frente al $(1+1)$ -ES.

CONCLUSIONES DE LA PRIMERA PARTE

7.1. RESUMEN DE LOS PRINCIPALES RESULTADOS

En esta primera práctica hemos implementado y comparado dos estrategias evolutivas fundamentales para la optimización de funciones reales. Ambas han demostrado ser capaces de minimizar la función Sphere en 30 dimensiones. La estrategia poblacional $(\mu + \lambda)$, con $\mu = 10$ y $\lambda = 30$, ha resultado especialmente estable y robusta.

8. ALGORITMO GENÉTICO BINARIO

8.1. DESCRIPCIÓN DEL PROBLEMA

A diferencia de las estrategias evolutivas de la primera parte, este apartado se centra en un algoritmo genético (AG) clásico con codificación *binaria*. Cada individuo de la población es un vector de 1000 bits donde cada elemento puede tomar el valor 0 o 1. El objetivo es **maximizar** la siguiente función objetivo sencilla pero ilustrativa:

$$\text{fitness}(x) = \sum_{i=1}^{1000} x_i$$

Dicho de otro modo, buscamos individuos que contengan el mayor número posible de unos. La codificación es puramente discreta; el dominio de búsqueda es el conjunto de todas las cadenas binarias de longitud 1000. La función es unimodal: existe un único óptimo global en la cadena formada por todos los unos, con valor de fitness 1000. No hay restricciones explícitas salvo la longitud fija del cromosoma. Por tanto, la dificultad no proviene de la existencia de mínimos locales sino del tamaño exponencial del espacio de búsqueda y de la necesidad de un muestreo eficiente para alcanzar el óptimo en un número razonable de generaciones.

8.2. REPRESENTACIÓN Y POBLACIÓN

Cada individuo se modela como un objeto *Individuo* que contiene un array de *numpy* de 1000 posiciones con valores 0/1 y un campo auxiliar *num_unos* con el número de bits a uno. La población inicial se genera **aleatoriamente**, asignando a cada posición de cada cromosoma un valor binario equiprobable. Para cumplir con el enunciado se utiliza un tamaño de población constante ($N = 100$) individuos. Este tamaño representa un compromiso entre diversidad genética y coste computacional: un tamaño mayor incrementa la exploración pero retrasa la convergencia, mientras que una población demasiado pequeña puede derivar en pérdida prematura de diversidad.

8.3. SELECCIÓN POR RANKING LINEAL

El algoritmo implementa una **selección por ranking lineal**. En cada generación se ordenan los individuos por su fitness de mayor a menor. La probabilidad de selección del individuo en la posición *i* se calcula mediante la fórmula de Baker para ranking lineal:

$$\text{prob} = (2 - S) / N + (2 * (S - 1) * (N - 1 - i)) / (N * (N - 1))$$

donde $N=100$ y $S \in [1,2]$ controla la presión selectiva (en nuestra implementación se tomó $S=1$). Esta estrategia impone que el mejor individuo no monopolice la reproducción (como ocurriría con la selección proporcional al fitness) y garantiza una suma de probabilidades igual a uno. Tras calcular estas probabilidades se seleccionan aleatoriamente 200 individuos (100 parejas) con reemplazamiento para participar en el cruce.

8.4. OPERADOR DE CRUCE (UN PUNTO)

Una vez seleccionadas las parejas, se aplica un **cruce de un punto**. Para cada pareja de padres se escoge un índice de cruce de forma uniforme y se crea un hijo concatenando las *k* primeras posiciones de un padre con el resto del otro. Se alterna aleatoriamente el sentido (izquierda/derecha) para mantener la diversidad. Dado que se generan 100 parejas, obtenemos 100 descendientes por generación.

El cruce es la fuente principal de recombinación y permite mezclar fragmentos de cromosomas que han demostrado ser buenos. Sin embargo, por sí mismo no puede introducir nueva información en bits nunca explorados; para ello es necesario el operador de mutación.

8.5. MUTACIÓN BIT A BIT Y VECTORIZACIÓN

El operador de mutación realiza un **flip** independiente en cada bit del hijo con una probabilidad baja, (0,1 %). Si un bit muta, su valor se invierte $\rightarrow$. Una implementación directa con bucles internos supondría 1000 comprobaciones por individuo y generación, lo que resulta ineficiente (hasta 500 millones de comparaciones en 5000 generaciones). Para reducir el coste, se recurre a la **vectorización** con NumPy: se genera un array de 1000 números aleatorios de golpe, se construye una máscara booleana para las posiciones donde la mutación ocurre y se invierten los bits sólo en esas posiciones. Además, se recalcula *num_unos* únicamente si la máscara contiene algún *True*. Estas optimizaciones trasladan el trabajo a rutinas compiladas y aceleran significativamente la simulación.

8.6. ELITISMO Y REEMPLAZO GENERACIONAL

Para evitar la pérdida de soluciones ya encontradas ("olvido catastrófico"), la versión final del algoritmo implementa **elitismo**. Antes de generar la nueva población se guarda una copia profunda del mejor individuo de la generación actual. Tras producir los hijos y aplicar la mutación, se reemplaza al peor hijo por el padre elitista. A continuación, la población de la siguiente generación se forma exclusivamente por los 100 hijos (incluyendo al elitista). Este mecanismo actúa como un *trinquete evolutivo* que garantiza que el fitness máximo nunca decrece de una generación a la siguiente y mejora la convergencia.

8.7. PSEUDOCÓDIGO DEL AG

A partir de la implementación se puede extraer el siguiente esquema de funcionamiento:

1. Inicializar una población P de $N = 100$ individuos con bits aleatorios.
 2. Calcular *num_unos* de cada individuo y almacenar el mejor individuo global.
 3. Para cada generación $g = 0, 1, \dots, 4999$:
 - a. **Selección:** ordenar P por *num_unos* y asignar probabilidades mediante ranking lineal. Seleccionar 200 índices (100 parejas) con reemplazamiento según dichas probabilidades.
 - b. **Cruce:** para cada pareja, elegir un punto de cruce y generar un hijo concatenando fragmentos alternados de los padres.
 - c. **Mutación vectorizada:** para cada hijo, generar una máscara booleana *mask* de longitud 1000 con probabilidad *p_m* y realizar:
$$\text{hijo.array}[\text{mask}] = 1 - \text{hijo.array}[\text{mask}]$$
- Si hay mutación, actualizar *hijo.num_unos*.

d. **Elitismo:** ordenar los hijos por fitness descendente y reemplazar el peor individuo por una copia del mejor padre de la generación anterior.

e. **Actualización:** sustituir la población P por el conjunto de hijos. Si el nuevo mejor hijo supera al mejor global, actualizar `mejor_individuo_global` y anotar la generación.

4. Terminar si `mejor_individuo_global.num_unos == 1000` o se alcanzan 5000 generaciones.

8.8. PARÁMETROS UTILIZADOS

Los valores elegidos para los experimentos son los especificados en el enunciado, con algunos ajustes justificados:

- **Tamaño de población:** $N = 100$ individuos (padres e hijos se gestionan dentro de este tamaño fijo).
- **Número máximo de generaciones:** 5000. El criterio de parada alternativo se produce si se encuentra un individuo con 1000 unos antes de ese límite.
- **Probabilidad de mutación:** $p_m = 0,001$, es decir, un 0,1 % de probabilidad de invertir cada bit en un hijo. Esta tasa baja equilibra la exploración y mantiene la estructura de buenas soluciones.
- **Presión selectiva del ranking:** $S = 1,5$, valor intermedio que evita tanto el elitismo extremo como la aleatoriedad excesiva.
- **Crossover:** de un punto, con elección aleatoria del punto y del orden de los fragmentos.

8.9. RESULTADOS Y GRÁFICAS DE EVOLUCIÓN

La figura siguiente muestra la evolución del fitness a lo largo de las generaciones. Se representan dos curvas: el número de unos del **mejor individuo de cada generación** y el **promedio de la población**. La simulación se detuvo cuando la cadena óptima de 1000 unos fue encontrada en la generación 2062, mucho antes de alcanzar el límite de 5000 generaciones.

Obsérvese cómo el elitismo hace que la curva del mejor fitness sea monótonamente creciente, mientras que el promedio sube de forma más suave. Al inicio ($g < 200$) la mejora es rápida debido a la alta diversidad inicial y a la recombinación efectiva. Posteriormente, la curva se aplana a medida que se aproxima al máximo y las mutaciones se vuelven la única fuente de innovación.

8.10. ANÁLISIS Y DISCUSIÓN DE LOS RESULTADOS

La dinámica observada en la gráfica es coherente con la teoría de algoritmos genéticos:

- **Fase de exploración inicial:** la población aleatoria parte con un número medio de unos alrededor de 500. La selección por ranking favorece a los individuos ligeramente superiores, pero mantiene diversidad suficiente. Los cruces de un punto combinan subsecuencias de bits a uno procedentes

de distintos padres, produciendo saltos significativos en el mejor fitness durante las primeras decenas de generaciones.

- **Fase de explotación:** conforme el número de unos medio se acerca a 900, gran parte de la población comparte los mismos patrones de bits. El cruce deja de aportar variaciones útiles y son las mutaciones puntuales las que permiten aumentar de 990 a 1000 unos. La introducción de elitismo evita retrocesos y asegura que los logros se acumulen. Sin elitismo, el fitness del mejor individuo oscilaría debido a la desaparición ocasional de buenas soluciones.
- **Efecto de la probabilidad de mutación:** una tasa demasiado alta destruiría la información acumulada; una tasa muy baja haría imposible alcanzar los últimos bits a uno. El valor $p_m = 0,001$ resulta un compromiso razonable para cadenas largas.
- **Ventajas de la selección por ranking:** a diferencia de la selección por ruleta, el ranking lineal evita la pérdida temprana de diversidad y permite un control fino de la presión selectiva mediante el parámetro S . Esto es especialmente importante en problemas con un único óptimo global.

El resultado final demuestra que el algoritmo, en su versión optimizada, es capaz de encontrar la solución óptima de forma fiable dentro de unas pocas centenas de generaciones. La vectorización y el elitismo reducen drásticamente el tiempo de ejecución y garantizan una convergencia suave. Como punto de comparación, implementaciones sin estas mejoras presentan curvas de fitness con "dientes de sierra" y, en algunos casos, no alcanzan 1000 unos antes de la generación 5000.

